

PomPom

Typing Rules and Metatheory

A mathematical presentation of the repository formalization

Source snapshot: 5 September 2026

Abstract

This document presents the minimal PomPom calculus using mathematical inference rules: dependent functions and pairs, enumerations, indexed descriptions, inductive families, and signature refinements. It also states the principal metatheoretic results and the outstanding proof obligations. The authoritative source in this checkout is the Rocq/Coq development `progress/TypeRules.v` and its companion proof modules; no Lean source is present. The notation is adapted for reading, while synthesis, checking, reduction, and assumptions remain distinct.

Contents

1	Notation and scope	2
2	Contexts, functions, and pairs	2
3	Unit, enumerations, and lists	3
4	Indexed descriptions and inductive families	4
5	Signature refinements and subtyping	5
6	Label side conditions and conversion	6
7	Computation	8
8	Metatheory and proof status	9

1 Notation and scope

Contexts Γ contain variable declarations. We write named binders for readability; the implementation uses de Bruijn indices. Thus $B[a/x]$ denotes capture-avoiding substitution, and weakening beneath a binder is implicit in named notation. In the implementation, variable n with stored type A has type $\text{lift}(n+1, 0, A)$.

Notation	Meaning in the formalization
$\Gamma \vdash t \Rightarrow A$	synth : synthesize a type
$\Gamma \vdash t \Leftarrow A$	check : check against a type
$\Gamma \vdash A \sqsubseteq B$	sub : coercion-free subtyping
$t \equiv u$	conv : untyped conversion, including φ
$t \mapsto u$	step : computation with inspected-position congruence
$t \Downarrow u$	eval : zero or more computation steps
$\text{Val}(t)$	value : designated weak-head value forms

Here $t \Downarrow u$ does not assert that u is a normal form. Moreover, a designated value can still step inside a pair or constructor. Progress is consequently an inclusive disjunction.

Universes are predicative: $*$ = Set_0 , Type = Set_1 . Functions and pairs exist at every level; subtyping includes cumulativity. $A \rightarrow B$ and $A \times B$ abbreviate nondependent products. In particular, $\text{IDesc } I : \text{Set}_1$ stores codes with small parameter types, while their interpretations live in Set_0 .

Rules below retain the premises of the inductive definitions, including places where no separate formation premise is imposed. Conversion is a relation on raw terms; it is not a typed equality judgment. Rule names identify the corresponding source constructors. The notation π_k and switch_k records their typing universe; their raw syntax stores no k .

Source: `progress/TypeRules.v`: `term`, `lift`, `subst`, `wf`, `synth`, `check`, `sub`.

2 Contexts, functions, and pairs

$$\begin{array}{c}
\frac{}{\emptyset \text{ valid}} \text{WF-NIL} \quad \frac{\Gamma \text{ valid} \quad \Gamma \vdash A \Leftarrow \text{Set}_k}{\Gamma, x : A \text{ valid}} \text{WF-CONS} \quad \frac{\Gamma \text{ valid} \quad x : A \in \Gamma}{\Gamma \vdash x \Rightarrow A} \text{SY-VAR} \\
\\
\frac{\Gamma \text{ valid}}{\Gamma \vdash \text{Set}_k \Rightarrow \text{Set}_{k+1}} \text{SY-SORT} \quad \frac{\Gamma \vdash A \Leftarrow \text{Set}_k \quad \Gamma, x : A \vdash B \Leftarrow \text{Set}_k}{\Gamma \vdash \Pi x : A. B \Rightarrow \text{Set}_k} \text{SY-PI} \\
\\
\frac{\Gamma \vdash A \Leftarrow \text{Set}_k \quad \Gamma, x : A \vdash B \Leftarrow \text{Set}_k}{\Gamma \vdash \Sigma x : A. B \Rightarrow \text{Set}_k} \text{SY-SIGMA} \quad \frac{\Gamma, x : A \vdash b \Leftarrow B}{\Gamma \vdash \lambda x. b \Leftarrow \Pi x : A. B} \text{CH-LAM} \\
\\
\frac{\Gamma \vdash a \Leftarrow A \quad \Gamma \vdash b \Leftarrow B[a/x]}{\Gamma \vdash (a, b) \Leftarrow \Sigma x : A. B} \text{CH-PAIR}
\end{array}$$

Synthesis exposes a function or pair type by evaluation before elimination.

$$\begin{array}{c}
\frac{\Gamma \vdash f \Rightarrow C \quad C \Downarrow \Pi x : A. B \quad \Gamma \vdash a \Leftarrow A}{\Gamma \vdash f a \Rightarrow B[a/x]} \text{SY-APP} \quad \frac{\Gamma \vdash p \Rightarrow C \quad C \Downarrow \Sigma x : A. B}{\Gamma \vdash \pi_0 p \Rightarrow A} \text{SY-FST} \\
\\
\frac{\Gamma \vdash p \Rightarrow C \quad C \Downarrow \Sigma x : A. B}{\Gamma \vdash \pi_1 p \Rightarrow B[\pi_0 p/x]} \text{SY-SND}
\end{array}$$

Checking also has declarative elimination rules. The eliminated type is chosen as part of the derivation and must be well formed. These rules allow elimination of checked-only terms such as a

literal lambda or pair.

$$\frac{\Gamma \vdash \Pi x : A.B \Leftarrow \text{Set}_k \quad \Gamma \vdash f \Leftarrow \Pi x : A.B \quad \Gamma \vdash a \Leftarrow A}{\Gamma \vdash f a \Leftarrow B[a/x]} \text{CH-APP}$$

$$\frac{\Gamma \vdash \Sigma x : A.B \Leftarrow \text{Set}_k \quad \Gamma \vdash p \Leftarrow \Sigma x : A.B}{\Gamma \vdash \pi_0 p \Leftarrow A} \text{CH-FST}$$

$$\frac{\Gamma \vdash \Sigma x : A.B \Leftarrow \text{Set}_k \quad \Gamma \vdash p \Leftarrow \Sigma x : A.B}{\Gamma \vdash \pi_1 p \Leftarrow B[\pi_0 p/x]} \text{CH-SND}$$

2.1 Conversion and subsumption

$$\frac{\Gamma \vdash t \Rightarrow A \quad A \equiv B}{\Gamma \vdash t \Leftarrow B} \text{CH-CONV} \qquad \frac{\Gamma \vdash t \Rightarrow A \quad \Gamma \vdash A \sqsubseteq B}{\Gamma \vdash t \Leftarrow B} \text{CH-SUB}$$

$$\frac{A \equiv B \quad \Gamma \vdash t \Leftarrow B}{\Gamma \vdash t \Leftarrow A} \text{CH-EXPAND}$$

These rules retype the same term; no runtime coercion is inserted. In particular, CH-CONV does not add a separate premise asserting that B inhabits a universe.

Source: `progress/TypeRules.v`: `wf_nil` through `sy_snd`; `ch_conv` through `ch_snd`.

3 Unit, enumerations, and lists

$$\begin{array}{ccc} \frac{\Gamma \text{ valid}}{\Gamma \vdash \mathbf{1} \Rightarrow \text{Set}_k} \text{SY-UNITT} & \frac{\Gamma \text{ valid}}{\Gamma \vdash \text{unit} \Rightarrow \mathbf{1}} \text{SY-UNIT} & \frac{\Gamma \text{ valid}}{\Gamma \vdash \text{Uld} \Rightarrow \text{Set}_0} \text{SY-UID} \\ \\ \frac{\Gamma \text{ valid}}{\Gamma \vdash \text{EnumU} \Rightarrow \text{Set}_0} \text{SY-ENUMU} & \frac{\Gamma \text{ valid}}{\Gamma \vdash 's \Rightarrow \text{Uld}} \text{SY-TAG} & \frac{\Gamma \text{ valid}}{\Gamma \vdash \text{nilE} \Rightarrow \text{EnumU}} \text{SY-NILE} \\ \\ \frac{\Gamma \vdash \ell \Leftarrow \text{Uld} \quad \Gamma \vdash E \Leftarrow \text{EnumU}}{\Gamma \vdash \text{consE } \ell E \Rightarrow \text{EnumU}} \text{SY-CONSE} & \frac{\Gamma \vdash E \Leftarrow \text{EnumU}}{\Gamma \vdash \text{EnumT } E \Rightarrow \text{Set}_0} \text{SY-ENUMT} \\ \\ \frac{\Gamma \vdash \ell \Leftarrow \text{Uld} \quad \Gamma \vdash E \Leftarrow \text{EnumU}}{\Gamma \vdash 0_E \Leftarrow \text{EnumT}(\text{consE } \ell E)} \text{CH-EZERO} & \frac{\Gamma \vdash \ell \Leftarrow \text{Uld} \quad \Gamma \vdash n \Leftarrow \text{EnumT } E}{\Gamma \vdash 1_E + n \Leftarrow \text{EnumT}(\text{consE } \ell E)} \text{CH-ESUCC} \end{array}$$

Every source string s yields a tag; the core imposes no identifier-validation premise. Constructor labels are ambient positions: $\text{Label } E := \text{EnumT } E$. Printable-name resolution belongs to elaboration.

$$\begin{array}{c} \frac{\Gamma \vdash E \Leftarrow \text{EnumU} \quad \Gamma \vdash P \Leftarrow \text{EnumT } E \rightarrow \text{Set}_k}{\Gamma \vdash \pi_k E P \Rightarrow \text{Set}_k} \text{SY-EPI} \\ \\ \frac{\Gamma \vdash E \Leftarrow \text{EnumU} \quad \Gamma \vdash P \Leftarrow \text{EnumT } E \rightarrow \text{Set}_k \quad \Gamma \vdash p \Leftarrow \pi_k E P \quad \Gamma \vdash e \Leftarrow \text{EnumT } E}{\Gamma \vdash \text{switch}_k E P p e \Rightarrow P e} \text{SY-SWITCH} \\ \\ \frac{\Gamma \vdash A \Leftarrow \text{Set}_0}{\Gamma \vdash \text{List } A \Rightarrow \text{Set}_0} \text{SY-LIST} \qquad \frac{\Gamma \vdash A \Leftarrow \text{Set}_0}{\Gamma \vdash []_A \Rightarrow \text{List } A} \text{SY-LNIL} \\ \\ \frac{\Gamma \vdash A \Leftarrow \text{Set}_0 \quad \Gamma \vdash a \Leftarrow A \quad \Gamma \vdash l \Leftarrow \text{List } A}{\Gamma \vdash a ::_A l \Rightarrow \text{List } A} \text{SY-LCONS} \end{array}$$

Lists are primitive stand-ins for library datatypes in this minimal specification. Unit formation at every level accommodates the reduction $\pi_k \text{nilE } P \mapsto \mathbf{1}$.

Source: `progress/TypeRules.v`: `sy_unitT` through `sy_switch`; `sy_list`, `sy_lnil`, `sy_lcons`; `ch_ezero`, `ch_esucc`.

4 Indexed descriptions and inductive families

4.1 Codes and interpretation

Description codes describe one layer of an indexed inductive family. Quoted constructors distinguish codes from the types they denote.

$$\begin{array}{c}
\frac{\Gamma \vdash I \Leftarrow \text{Set}_0}{\Gamma \vdash \text{IDesc } I \Rightarrow \text{Set}_1} \text{SY-IDESC} \quad \frac{\Gamma \vdash i \Leftarrow I}{\Gamma \vdash \text{'var } i \Leftarrow \text{IDesc } I} \text{CH-IVAR} \quad \frac{\Gamma \text{ valid}}{\Gamma \vdash \text{'1} \Leftarrow \text{IDesc } I} \text{CH-I1} \\
\\
\frac{\Gamma \vdash D \Leftarrow \text{IDesc } I \quad \Gamma \vdash D' \Leftarrow \text{IDesc } I}{\Gamma \vdash D' \times D' \Leftarrow \text{IDesc } I} \text{CH-IPROD} \\
\\
\frac{\Gamma \vdash A \Leftarrow \text{Set}_0 \quad \Gamma \vdash T \Leftarrow A \rightarrow \text{IDesc } I}{\Gamma \vdash \text{' } \Pi A T \Leftarrow \text{IDesc } I} \text{CH-IP1} \\
\\
\frac{\Gamma \vdash A \Leftarrow \text{Set}_0 \quad \Gamma \vdash T \Leftarrow A \rightarrow \text{IDesc } I}{\Gamma \vdash \text{' } \Sigma A T \Leftarrow \text{IDesc } I} \text{CH-ISIG} \\
\\
\frac{\Gamma \vdash E \Leftarrow \text{EnumU} \quad \Gamma \vdash T \Leftarrow \text{EnumT } E \rightarrow \text{IDesc } I}{\Gamma \vdash \text{' } \sigma E T \Leftarrow \text{IDesc } I} \text{CH-ICHOICE} \\
\\
\frac{\Gamma \vdash I \Leftarrow \text{Set}_0 \quad \Gamma \vdash D \Leftarrow \text{IDesc } I \quad \Gamma \vdash X \Leftarrow I \rightarrow \text{Set}_0}{\Gamma \vdash \llbracket D \rrbracket X \Rightarrow \text{Set}_0} \text{SY-INTERP}
\end{array}$$

4.2 Fixed points and induction

For compactness, the following abbreviations stand for the displayed checking premises, not additional judgments of the calculus:

$$\begin{aligned}
\mathcal{R}_\Gamma(I, R) &:= \Gamma \vdash I \Leftarrow \text{Set}_0 \wedge \Gamma \vdash R \Leftarrow I \rightarrow \text{IDesc } I, \\
\mathcal{D}_\Gamma(I, D, X) &:= \Gamma \vdash I \Leftarrow \text{Set}_0 \wedge \Gamma \vdash D \Leftarrow \text{IDesc } I \wedge \Gamma \vdash X \Leftarrow I \rightarrow \text{Set}_0.
\end{aligned}$$

$$\frac{\mathcal{R}_\Gamma(I, R)}{\Gamma \vdash \mu^I R \Rightarrow I \rightarrow \text{Set}_0} \text{SY-MUI} \quad \frac{\mathcal{R}_\Gamma(I, R) \quad \Gamma \vdash i \Leftarrow I \quad \Gamma \vdash xs \Leftarrow \llbracket Ri \rrbracket (\mu^I R)}{\Gamma \vdash \text{in } xs \Leftarrow \mu^I R i} \text{CH-IN-MUI}$$

Define the motive type and the type of a family of induction hypotheses by

$$\mathcal{P}(I, X) := (\Sigma i : I. X i) \rightarrow \text{Set}_0, \quad H(I, X, P) := \Pi i : I. \Pi x : X i. P(i, x).$$

$$\frac{\mathcal{D}_\Gamma(I, D, X) \quad \Gamma \vdash xs \Leftarrow \llbracket D \rrbracket X \quad \Gamma \vdash P \Leftarrow \mathcal{P}(I, X)}{\Gamma \vdash \text{iAll } D X xs P \Rightarrow \text{Set}_0} \text{SY-IALL}$$

$$\frac{\mathcal{D}_\Gamma(I, D, X) \quad \Gamma \vdash P \Leftarrow \mathcal{P}(I, X) \quad \Gamma \vdash h \Leftarrow H(I, X, P) \quad \Gamma \vdash xs \Leftarrow \llbracket D \rrbracket X}{\Gamma \vdash \text{hyps } D X P h xs \Rightarrow \text{iAll } D X xs P} \text{SY-HYPS}$$

The induction-step type is

$$\mathcal{S}(I, R, P) := \Pi i : I. \Pi xs : \llbracket Ri \rrbracket (\mu^I R). \text{iAll } (R i) (\mu^I R) xs P \rightarrow P(i, \text{in } xs).$$

$$\frac{\mathcal{R}_\Gamma(I, R) \quad \Gamma \vdash P \Leftarrow \mathcal{P}(I, \mu^I R) \quad \Gamma \vdash s \Leftarrow \mathcal{S}(I, R, P) \quad \Gamma \vdash i \Leftarrow I \quad \Gamma \vdash x \Leftarrow \mu^I R i}{\Gamma \vdash \text{iinduction } R P s i x \Rightarrow P(i, x)} \text{SY-IND}$$

Source: `progress/TypeRules.v`: `sy_idesc` through `sy_ind`; `ch_ivar` through `ch_in_mui`.

5 Signature refinements and subtyping

5.1 Signatures as ordinary pairs

A signature packages a branch-description function together with enabled labels:

$$\begin{aligned} \text{Sig } I E &:= (\text{EnumT } E \rightarrow \text{IDesc } I) \times \text{List}(\text{Label } E), \\ \{T :: \Phi\} &:= (T, \Phi), \\ \text{branches}(s) &:= \pi_0 s, & \text{labels}(s) &:= \pi_1 s, \\ \text{Full}_E(s) &:= \sigma E (\text{branches}(s)), \\ \text{Carrier}_E(S) &:= \mu^I (\lambda i. \text{Full}_E(S i)). \end{aligned}$$

Write $B_S(i, c) := \text{branches}(S i) c$ and $L_S(i) := \text{labels}(S i)$. The family-formation abbreviation is

$$\mathcal{F}_\Gamma(I, E, S) : \Longleftrightarrow \Gamma \vdash I \Leftarrow \text{Set}_0 \wedge \Gamma \vdash E \Leftarrow \text{EnumU} \wedge \Gamma \vdash S \Leftarrow \Pi i : I. \text{Sig } I E.$$

$$\frac{\mathcal{F}_\Gamma(I, E, S)}{\Gamma \vdash \mu^s S \Rightarrow I \rightarrow \text{Set}_0} \text{SY-MUS}$$

$$\frac{\Gamma \vdash c \Leftarrow \text{Label } E \quad \mathcal{F}_\Gamma(I, E, S) \quad \Gamma \vdash i \Leftarrow I \quad L_S(i) \Downarrow \Phi \quad c \in_s \Phi \quad \Gamma \vdash xs \Leftarrow \llbracket B_S(i, c) \rrbracket (\text{Carrier}_E(S))}{\Gamma \vdash \text{in}(c, xs) \Leftarrow \mu^s S i} \text{CH-IN-SIG}$$

The recursive payload uses the full carrier. The enabled list restricts the outer constructor; no membership evidence is stored in the term.

5.2 Subtyping

$$\frac{A \equiv B}{\Gamma \vdash A \sqsubseteq B} \text{SU-CONV} \quad \frac{\Gamma \vdash A \sqsubseteq B \quad \Gamma \vdash B \sqsubseteq C}{\Gamma \vdash A \sqsubseteq C} \text{SU-TRANS} \quad \frac{j \leq k}{\Gamma \vdash \text{Set}_j \sqsubseteq \text{Set}_k} \text{SU-SORT}$$

$$\frac{\Gamma \vdash A' \sqsubseteq A \quad \Gamma, x : A' \vdash B \sqsubseteq B'}{\Gamma \vdash \Pi x : A. B \sqsubseteq \Pi x : A'. B'} \text{SU-PI} \quad \frac{\mathcal{F}_\Gamma(I, E, S) \quad \Gamma \vdash i \Leftarrow I}{\Gamma \vdash \mu^s S i \sqsubseteq \text{Carrier}_E(S) i} \text{SU-FORGET}$$

$$\frac{\mathcal{F}_\Gamma(I, E, S_1) \quad \mathcal{F}_\Gamma(I, E, S_2) \quad \Gamma \vdash i \Leftarrow I \quad (\lambda j. \text{branches}(S_1 j)) \equiv (\lambda j. \text{branches}(S_2 j)) \quad L_{S_1}(i) \Downarrow \Phi_1 \quad L_{S_2}(i) \Downarrow \Phi_2 \quad \Phi_1 \subseteq_s \Phi_2}{\Gamma \vdash \mu^s S_1 i \sqsubseteq \mu^s S_2 i} \text{SU-SIG}$$

In SU-SIG, the shared branch descriptions are invariant up to conversion. Inclusion of enabled labels alone does not permit changing their payload types. The duplicated formation premises in the abbreviation refer to the same I and E .

5.3 Case analysis

Let $bs = [(c_1, N_1), \dots, (c_m, N_m)]$ and $\Psi = [c_1, \dots, c_m]$. Each branch body binds exactly one payload variable. The result type Q is independent of that variable.

$$\frac{\begin{array}{c} \mathcal{F}_\Gamma(I, E, S) \quad \Gamma \vdash i \Leftarrow I \\ \Gamma \vdash M \Leftarrow \mu^s S i \quad \Gamma \vdash Q \Leftarrow \text{Set}_k \quad \text{distinct}_{\equiv}(\Psi) \quad L_S(i) \Downarrow \Phi \quad \text{covers}(\Psi, \Phi) \\ \forall r, \quad \Gamma \vdash c_r \Leftarrow \text{Label } E \quad \forall r, \quad \Gamma, xs : \llbracket B_S(i, c_r) \rrbracket (\text{Carrier}_E(S)) \vdash N_r \Leftarrow Q \end{array}}{\Gamma \vdash \text{case } M \text{ of } Q \{c_r xs \Rightarrow N_r\}_{r=1}^m \Rightarrow Q} \text{SY-CASE}$$

Coverage requires every enabled label to occur among the clauses. Additional distinct clauses are allowed: reverse inclusion is not a typing premise. For zero clauses the two per-clause premises are vacuous. In de Bruijn notation the branch result is $\text{lift}(1, 0, Q)$.

Source: `progress/TypeRules.v`: `Sig`, `Carrier`, `sy_mus`, `sy_case`, `ch_in_sig`, `check_branches`, `sub`.

6 Label side conditions and conversion

The enabled labels are terms representing lists. The relations below inspect those lists by evaluation; they are meta-level relations, not library propositions stored in programs. Type annotations A on list constructors are implicit when immaterial.

6.1 Membership, inclusion, and coverage

$$\begin{array}{c} \frac{\Phi \Downarrow c' :: \Phi' \quad c \equiv c'}{c \in_s \Phi} \text{SM-HERE} \quad \frac{\Phi \Downarrow c' :: \Phi' \quad c \in_s \Phi'}{c \in_s \Phi} \text{SM-THERE} \quad \frac{\Phi \equiv \Psi}{\text{tail}(\Phi, \Psi)} \text{STL-HERE} \\ \\ \frac{\Psi \Downarrow c :: \Psi' \quad \text{tail}(\Phi, \Psi')}{\text{tail}(\Phi, \Psi)} \text{STL-THERE} \quad \frac{\Phi \Downarrow []_A}{\Phi \subseteq_s \Psi} \text{SI-NIL} \\ \\ \frac{\Phi \Downarrow c :: \Phi' \quad c \in_s \Psi \quad \Phi' \subseteq_s \Psi}{\Phi \subseteq_s \Psi} \text{SI-CONS} \\ \\ \frac{\Phi \Downarrow \Phi_n \quad \text{neutral}(\Phi_n) \quad \text{tail}(\Phi_n, \Psi)}{\Phi \subseteq_s \Psi} \text{SI-NEUTRAL} \quad \frac{\Phi \Downarrow []_A}{\text{covers}(\Psi, \Phi)} \text{COV-NIL} \\ \\ \frac{\Phi \Downarrow c :: \Phi' \quad \exists d \in \Psi, c \equiv d \quad \text{covers}(\Psi, \Phi')}{\text{covers}(\Psi, \Phi)} \text{COV-CONS} \quad \frac{}{\text{distinct}_{\equiv}([])} \text{DT-NIL} \\ \\ \frac{\forall d \in cs, \neg(c \equiv d) \quad \text{distinct}_{\equiv}(cs)}{\text{distinct}_{\equiv}(c :: cs)} \text{DT-CONS} \end{array}$$

Coverage has no neutral-tail rule. A list that cannot expose the required constructors supplies no evidence of coverage. The source's neutral forms are exactly

$$n ::= x \mid n a \mid \pi_0 n \mid \pi_1 n \mid \text{switch}_k E P p n \mid \text{iinduction } R P s i n \mid \text{case } n \text{ of } Q \text{ } bs.$$

Source: `progress/TypeRules.v`: `neutral`, `spine_mem`, `spine_tail`, `spine_incl`, `covers`, `distinct`.

6.2 Positive evidence of a dead description

Write $\text{Dead}(D)$ for `desc_against D`, the conservative syntactic predicate used for pruning. Its intended uninhabitedness interpretation remains a metatheoretic obligation. Put $T^+ := \lambda e. T(1_E +$

e).

$$\begin{array}{c}
\frac{D \Downarrow' \sigma E T \quad E \Downarrow \text{nil} E}{\text{Dead}(D)} \text{AG-CHOICE-NIL} \qquad \frac{D \Downarrow A' \times B \quad \text{Dead}(A)}{\text{Dead}(D)} \text{AG-PROD-LEFT} \\
\\
\frac{D \Downarrow A' \times B \quad \text{Dead}(B)}{\text{Dead}(D)} \text{AG-PROD-RIGHT} \\
\\
\frac{D \Downarrow' \sigma E T \quad E \Downarrow \text{cons} E \ell E' \quad \text{Dead}(T 0_E) \quad \text{Dead}(' \sigma E' T^+)}{\text{Dead}(D)} \text{AG-CHOICE-CONS}
\end{array}$$

There is no dead-description constructor for $'\text{var}$, $'1$, $'\Pi$, or $'\Sigma$. Failure to evaluate is never evidence of emptiness.

6.3 Pruning enabled labels

Write $\Phi \rightsquigarrow_{S,i} \Psi$ for `spine_phi S i Phi Psi`.

$$\begin{array}{c}
\frac{\Phi \Downarrow []_A}{\Phi \rightsquigarrow_{S,i} []_A} \text{SPH-NIL} \qquad \frac{\Phi \Downarrow c ::_A \Phi' \quad \Phi' \rightsquigarrow_{S,i} \Psi'}{\Phi \rightsquigarrow_{S,i} c ::_A \Psi'} \text{SPH-KEEP} \\
\\
\frac{\Phi \Downarrow c ::_A \Phi' \quad \text{Dead}(B_S(i, c)) \quad \Phi' \rightsquigarrow_{S,i} \Psi'}{\Phi \rightsquigarrow_{S,i} \Psi'} \text{SPH-DROP} \\
\\
\frac{\Phi \Downarrow \Phi_n \quad \text{neutral}(\Phi_n)}{\Phi \rightsquigarrow_{S,i} \Phi_n} \text{SPH-NEUTRAL}
\end{array}$$

Pruning can retain a dead label, so it need not be exhaustive. A neutral tail is retained.

6.4 Definitional conversion

Conversion is the smallest equivalence and congruence containing computation, function eta, and the signature rule below. Congruence applies to every term former, including case labels and bodies.

$$\begin{array}{c}
\frac{t \mapsto u}{t \equiv u} \text{CV-STEP} \qquad \frac{}{t \equiv t} \text{CV-REFL} \qquad \frac{t \equiv u}{u \equiv t} \text{CV-SYM} \qquad \frac{t \equiv u \quad u \equiv v}{t \equiv v} \text{CV-TRANS} \\
\\
\frac{x \notin \text{FV}(f)}{(\lambda x. f x) \equiv f} \text{CV-ETA} \qquad \frac{t_1 \equiv u_1 \quad \dots \quad t_m \equiv u_m}{F(t_1, \dots, t_m) \equiv F(u_1, \dots, u_m)} \text{CONGRUENCE} \\
\\
\frac{L_{S_1}(i) \Downarrow \Phi_1 \quad L_{S_2}(i) \Downarrow \Phi_2 \quad \Phi_1 \rightsquigarrow_{S_1,i} \Psi_1 \quad \Phi_2 \rightsquigarrow_{S_2,i} \Psi_2 \quad \Psi_1 \equiv \Psi_2}{\mu^s S_1 i \equiv \mu^s S_2 i} \text{CV-PHI}
\end{array}$$

The congruence schema abbreviates the individual `cv_*` constructors, with binding structure preserved. $\text{Eq-}\varphi$ equates refinement types at an instance; it does not identify the signature pairs themselves. Consequently their label projections remain observable as ordinary terms. There are no context or typing premises on CV-PHI in the raw relation.

Source: `progress/TypeRules.v: desc_against, spine_phi, conv.`

7 Computation

7.1 Functions, pairs, enumerations, and interpretation

Let $P^+ := \lambda e. P(1_E + e)$. The primitive computation rules are

$$\begin{aligned} (\lambda x. b) a &\mapsto b[a/x], & \pi_0(a, b) &\mapsto a, \quad \pi_1(a, b) \mapsto b, \\ \pi_k \text{ nilE } P &\mapsto \mathbf{1}, \\ \pi_k (\text{consE } \ell E) P &\mapsto P 0_E \times \pi_k E P^+, \\ \text{switch}_k (\text{consE } \ell E) P (p_0, ps) 0_E &\mapsto p_0, \\ \text{switch}_k (\text{consE } \ell E) P (p_0, ps) (1_E + n) &\mapsto \text{switch}_k E P^+ ps n. \end{aligned}$$

$$\begin{aligned} \llbracket \text{'var } i \rrbracket X &\mapsto X i, \\ \llbracket \text{'1} \rrbracket X &\mapsto \mathbf{1}, \\ \llbracket A' \times B \rrbracket X &\mapsto \llbracket A \rrbracket X \times \llbracket B \rrbracket X, \\ \llbracket \text{'}\Pi A T \rrbracket X &\mapsto \Pi a : A. \llbracket T a \rrbracket X, \\ \llbracket \text{'}\Sigma A T \rrbracket X &\mapsto \Sigma a : A. \llbracket T a \rrbracket X, \\ \llbracket \text{'}\sigma E T \rrbracket X &\mapsto \Sigma e : \text{EnumT } E. \llbracket T e \rrbracket X. \end{aligned}$$

7.2 Induction hypotheses

In this subsection abbreviate $\mathcal{A}(D, x) := \text{iAll } D X x P$ and $\mathcal{H}(D, x) := \text{hyps } D X P h x$, with X, P, h fixed.

$$\begin{aligned} \mathcal{A}(\text{'var } j, x) &\mapsto P(j, x), & \mathcal{H}(\text{'var } j, x) &\mapsto h j x, \\ \mathcal{A}(\text{'1}, \text{unit}) &\mapsto \mathbf{1}, & \mathcal{H}(\text{'1}, \text{unit}) &\mapsto \text{unit}, \\ \mathcal{A}(A' \times B, (a, b)) &\mapsto \mathcal{A}(A, a) \times \mathcal{A}(B, b), \\ \mathcal{H}(A' \times B, (a, b)) &\mapsto (\mathcal{H}(A, a), \mathcal{H}(B, b)), \\ \mathcal{A}(\text{'}\Pi A T, f) &\mapsto \Pi a : A. \mathcal{A}(T a, f a), \\ \mathcal{H}(\text{'}\Pi A T, f) &\mapsto \lambda a. \mathcal{H}(T a, f a), \\ \mathcal{A}(\text{'}\Sigma A T, (a, x)) &\mapsto \mathcal{A}(T a, x), & \mathcal{H}(\text{'}\Sigma A T, (a, x)) &\mapsto \mathcal{H}(T a, x), \\ \mathcal{A}(\text{'}\sigma E T, (e, x)) &\mapsto \mathcal{A}(T e, x), & \mathcal{H}(\text{'}\sigma E T, (e, x)) &\mapsto \mathcal{H}(T e, x). \end{aligned}$$

Writing $h := \lambda j. \lambda y. \text{iinduction } R P s j y$, induction computes by

$$\text{iinduction } R P s i (\text{in } xs) \mapsto s i xs (\text{hyps } (R i) (\mu^I R) P h xs).$$

7.3 Case selection and evaluation positions

Write $c \Downarrow_{\text{pos}} n$ for the syntactic predicate `enum_pos`: $0_E \Downarrow_{\text{pos}} 0$ and $c \Downarrow_{\text{pos}} n$ implies $(1_E + c) \Downarrow_{\text{pos}} (n + 1)$. This notation does not denote an evaluation relation.

If the zero-based clause $bs[k] = (c_k, N_k)$ satisfies $c_k \Downarrow_{\text{pos}} n$ and $a \Downarrow_{\text{pos}} n$, and every earlier clause label has a canonical position different from n , then

$$\text{case } (\text{in}(a, xs)) \text{ of } Q bs \mapsto N_k[xs/x].$$

Thus raw case selection uses the first matching position. For a typed case, distinctness ensures uniqueness.

The relation is closed under the following one-hole computation contexts:

$$\mathcal{E} ::= [] a \mid \pi_0[] \mid \pi_1[] \mid \pi_k[] P$$

$$\begin{aligned}
& | \text{switch}_k [] P p e \mid \text{switch}_k E P [] e \mid \text{switch}_k E P p [] \\
& | [[]] X \mid \text{iAll} [] X x s P \mid \text{iAll} D X [] P \\
& | \text{hyps} [] X P h x s \mid \text{hyps} D X P h [] \\
& | \text{iinduction} R P s i [] \mid \text{case} [] \text{of } Q \text{ } b s \\
& | \text{case } M \text{ of } Q (b s_1 ++ ([], b) :: b s_2) \\
& | 1_E + [] \mid \text{in} [] \mid ([], b) \mid (a, []).
\end{aligned}$$

That is, $t \mapsto u$ implies $\mathcal{E}[t] \mapsto \mathcal{E}[u]$. This is a relation and does not impose a total evaluation order between the available positions. Evaluation $\Downarrow$ is its reflexive transitive closure. Fixed-point heads have no unfolding rule.

The value forms are sorts, Π , lambdas, Σ , pairs, unit type and unit, identifiers and tags, enumeration type and constructors, positional zero and successor, description type and codes, $\mu^I R$, $\mu^s S$, their one-argument applications, $\text{in } x$, and list types and constructors. Their components need not themselves be values.

Source: `progress/TypeRules.v`: `step`, `eval`, `enum_pos`, `value`.

8 Metatheory and proof status

This section separates unconditional results, proofs with explicit premises, and declarations made with `Conjecture`. A Coq conjecture is an assumption available to later proofs. A theorem ending in `Qed` can therefore still depend on a conjecture; its assumption audit determines the distinction.

8.1 Established structural results

Theorem 8.1 (Preservation when the type computes). *For every context Γ and terms t, A, A' ,*

$$\Gamma \vdash t \Leftarrow A \wedge A \mapsto A' \implies \Gamma \vdash t \Leftarrow A'.$$

This follows directly from conversion and CH-EXPAND; it does not require term preservation.

Source: `progress/TypeRules.v`: `preservation_type`.

Write $\text{whd}(t, h)$ when $t \Downarrow u$ for some term u with the stable head classification h used in `Progress.v`. These are the formalization's designated rigid heads; this notation does not classify every possible weak-head term.

Theorem 8.2 (Conversion preserves stable head classification).

$$t \equiv u \wedge \text{whd}(t, h_1) \wedge \text{whd}(u, h_2) \implies h_1 = h_2.$$

Source: `progress/Progress.v`: `conv_whd`.

Theorem 8.3 (Canonical positions are distinct under conversion).

$$c \equiv d \wedge c \Downarrow_{\text{pos}} n \wedge d \Downarrow_{\text{pos}} m \implies n = m.$$

Source: `progress/Progress.v`: `conv_pos`.

Theorem 8.4 (Coverage supplies a convertible label).

$$a \in_s L \wedge \text{covers}(\Psi, L) \implies \exists d \in \Psi, a \equiv d.$$

Source: `progress/Progress.v`: `spine_covered`.

Theorem 8.5 (A value application has a sort-classified type). *If $\emptyset \vdash f a \Leftarrow T$, $\text{Val}(f a)$, $T \equiv U$, and $\text{whd}(U, h)$, then $h = \text{HSort}$.*

Value applications here are the applied fixed-point forms designated by the definition of `value`.

Source: `progress/Progress.v`: `muapp_sort`.

8.2 Progress and the remaining signature interface

Define the property

$$\text{Progress} : \Longleftrightarrow \forall t, A, \quad \emptyset \vdash t \Leftarrow A \implies (\text{Val}(t) \vee \exists t', t \mapsto t').$$

Theorem 8.6 (Progress relative to signature canonical forms). *Assuming `canonical_forms_sig`, `Progress` holds.*

The theorem `progress_proved` has the displayed progress type and depends on the global conjecture `TypeRules.canonical_forms_sig`. It does not discharge that conjecture. The separate declaration `TypeRules.progress` remains a conjecture.

Source: `progress/Progress.v`: `progress_proved`.

A smaller, explicit interface also suffices. Let $|t|$ be `tsize t`, the source's structural size measure, and define

$$\text{BP}(N) : \Longleftrightarrow \forall t, A, \quad |t| \leq N \wedge \emptyset \vdash t \Leftarrow A \implies \text{Val}(t) \vee \exists t', t \mapsto t'.$$

Definition 8.7 (Signature tag transport, STT). For all $N, S_0, i_0, S_1, i_1, c, n, xs, X, \Psi$, assume

$$\begin{aligned} &\text{BP}(N), \quad |xs| \leq N, \\ &\emptyset \vdash xs \Leftarrow \llbracket B_{S_0}(i_0, c) \rrbracket X, \\ &\mu^s S_0 i_0 \equiv \mu^s S_1 i_1, \\ &c \Downarrow_{\text{pos}} n, \quad \forall d \in \Psi, \exists m, d \Downarrow_{\text{pos}} m, \\ &c \in_s L_{S_0}(i_0), \quad \text{covers}(\Psi, L_{S_1}(i_1)). \end{aligned}$$

Then

$$(\exists d \in \Psi, d \Downarrow_{\text{pos}} n) \quad \vee \quad (\exists xs', xs \mapsto xs').$$

Theorem 8.8 (Explicit transport implies progress). $\text{STT} \implies \text{Progress}$.

This implication has no global axiom dependency. Its premise `STT` is still unproved for arbitrary mixed conversion; having a closed proof of the implication does not supply its premise.

Source: `progress/SignatureProgressReduction.v`: `signature_tag_transport`, `progress_from_signature_tag_transport`.

Write $\equiv_f$ for `fconv`, the equivalence/congruence generated by core computation and function eta without `Eq-φ`. Core computation here includes the primitive eliminator rules, not just lambda beta reduction.

Theorem 8.9 (Tag transport without `Eq-φ`). *If*

$$\begin{aligned} &\mu^s S_0 i_0 \equiv_f \mu^s S_1 i_1, \quad c \Downarrow_{\text{pos}} n, \\ &\forall d \in \Psi, \exists m, d \Downarrow_{\text{pos}} m, \\ &c \in_s L_{S_0}(i_0), \quad \text{covers}(\Psi, L_{S_1}(i_1)), \end{aligned}$$

then $\exists d \in \Psi, d \Downarrow_{\text{pos}} n$.

Source: `progress/SignatureTagTransport.v`: `signature_live_tag_transport_luna`.

8.3 Outstanding metatheoretic statements

All conjectures in this subsection are declarations in `TypeRules.v`, not unconditional proved theorems in the integrated development.

Conjecture 8.10 (Normalization).

$$\emptyset \vdash t \Leftarrow A \implies \exists v, t \Downarrow v \wedge \text{Val}(v).$$

This is existence of a finite evaluation to a designated value; it is not a statement of strong normalization for all reduction paths.

Source: `progress/TypeRules.v: normalization`.

Conjecture 8.11 (Signature canonical forms). *If $\mathcal{F}_\emptyset(I, E, S)$, $\emptyset \vdash i \Leftarrow I$, and $\emptyset \vdash M \Leftarrow \mu^s S i$, then there exist c, xs, Φ such that*

$$\begin{aligned} M \Downarrow \text{in}(c, xs), \quad L_S(i) \Downarrow \Phi, \quad c \in_s \Phi, \\ \emptyset \vdash xs \Leftarrow \llbracket B_S(i, c) \rrbracket (\text{Carrier}_E(S)). \end{aligned}$$

The index-typing and family-formation premises are part of the statement and must be retained.

Source: `progress/TypeRules.v: canonical_forms_sig`.

Conjecture 8.12 (Preservation, or subject reduction).

$$\Gamma \vdash t \Leftarrow A \wedge t \mapsto t' \implies \Gamma \vdash t' \Leftarrow A.$$

Proposition 8.13 (Conditional multistep preservation). *Assuming preservation,*

$$\Gamma \vdash t \Leftarrow A \wedge t \Downarrow t' \implies \Gamma \vdash t' \Leftarrow A.$$

The multistep proposition is proved by induction on evaluation, with a global dependency on `TypeRules.preservation`.

Source: `progress/TypeRules.v: preservation, preservation_eval`.

Conjecture 8.14 (Consistency). *Let $\perp := \prod X : \text{Set}_0. X$. Then*

$$\forall t, \neg \emptyset \vdash t \Leftarrow \perp.$$

Conjecture 8.15 (Empty enumeration). $\forall t, \neg \emptyset \vdash t \Leftarrow \text{EnumT nilE}$.

Conjecture 8.16 (Empty enabled signature). *For every M, S, i, A ,*

$$\emptyset \vdash M \Leftarrow \mu^s S i \wedge L_S(i) \Downarrow []_A \implies \text{False}.$$

Source: `progress/TypeRules.v: consistency, consistency_enum, consistency_empty_sig`.

Conjecture 8.17 (Soundness of dead-description evidence). *For every I, D, X, t , if $\mathcal{D}_\emptyset(I, D, X)$ and $\text{Dead}(D)$, then*

$$\neg \emptyset \vdash t \Leftarrow \llbracket D \rrbracket X.$$

This claim is scoped to well-typed descriptions and carriers.

Source: `progress/TypeRules.v: against_sound`.

8.4 Proved limits of erasure arguments

The proof modules use auxiliary erasures. These are proof constructions, not coercions inserted by the typing rules. Let ε_φ denote `phi_erase`, and let $\longrightarrow_c^*$ denote `rtc cstep`, the auxiliary combined reduction used in the conversion proofs.

Theorem 8.18 (Erased sort endpoints do not reflect).

$$\neg \left(\forall t, j, \quad \varepsilon_\varphi(t) \longrightarrow_c^* \text{Set}_j \implies t \equiv \text{Set}_j \right).$$

The source supplies a concrete raw term `bad_t` whose erasure reaches `Set0`, while the original term is not convertible to any sort. This refutes an unrestricted erasure-reflection shortcut; it is not a counterexample to typed progress.

Source: `progress/ErasureCounterexample.v: bad_erased_sort_path, bad_not_conv_sort, phi_erase_sort_endpoint_not_reflectable`.

Theorem 8.19 (Branch erasure cannot recover enabled labels). *There are explicit raw signature families $S_{\text{full}}, S_{\text{empty}}$ and an index i such that*

$$\begin{aligned} 0_E &\in_s L_{S_{\text{full}}}(i), & \text{covers}([], L_{S_{\text{empty}}}(i)), \\ \text{join}_c(\varepsilon_b(\mu^s S_{\text{full}} i), \varepsilon_b(\mu^s S_{\text{empty}} i)), \\ \neg \exists d \in [], d \Downarrow_{\text{pos}} 0. \end{aligned}$$

Here ε_b is **branch_erase**, and join_c means that both terms reach a common term under $\longrightarrow_c^*$.

The witness families have the same constant unit description and respectively a singleton and an empty enabled list. Joinability after erasure supplies insufficient information to recover a live label. The theorem does not assert conversion of the original refined types.

Source: `progress/SignatureTagTransport.v`: `BranchErasureLabelGap.branch_erased_recovery_label_gap`.